

Estruturas de Dados em Java

Guia Completo para Iniciantes

Por que Estruturas de Dados?

Estruturas de dados são a forma como organizamos informações na memória. Escolher a estrutura certa é o que separa um código lento de um código rápido — e é exatamente o que as empresas avaliam em entrevistas técnicas.

Analogia: diferentes gavetas para diferentes coisas

Imagine um escritório:

- Pasta com índice = Array (acesso direto por posição)
- Fila de atendimento = Queue (primeiro a chegar, primeiro a ser atendido)
- Pilha de pratos = Stack (o último colocado é o primeiro retirado)
- Catálogo telefônico = HashMap (busca rápida por nome)
- Organograma da empresa = Árvore (hierarquia de nós)

Notação Big O — Medir a Eficiência

Antes de mergulhar nas estruturas, precisamos entender como medimos sua eficiência. A Notação Big O descreve como o tempo de execução cresce conforme o tamanho dos dados aumenta.

Notação	Nome	Exemplo prático	Velocidade
$O(1)$	Constante	Acessar array[5]	Instantâneo
$O(\log n)$	Logarítmica	Busca binária	Muito rápido
$O(n)$	Linear	Busca linear	Cresce com n
$O(n \log n)$	Log-Linear	Merge Sort	Aceitável
$O(n^2)$	Quadrática	Bubble Sort	Lento
$O(2^n)$	Exponencial	Subconjuntos	Muito lento

Como ler Big O:

- $O(1)$ = não importa o tamanho dos dados, sempre leva o mesmo tempo
- $O(n)$ = se dobrar os dados, dobra o tempo
- $O(n^2)$ = se dobrar os dados, quadruplica o tempo — evite sempre que possível

Nas tabelas de complexidade deste material: verde = rápido | amarelo = médio | vermelho = lento.

Estruturas Lineares

7.1 Arrays — A Base de Tudo

O array é a estrutura mais fundamental. Todos os elementos ficam em posições contíguas na memória, o que permite acesso instantâneo por índice — mas o tamanho é fixo e inserções/remoções no meio são custosas.

```
Índice:  [ 0 ] [ 1 ] [ 2 ] [ 3 ] [ 4 ]
Valor:   [ 10 ] [ 25 ] [ 37 ] [ 42 ] [ 58 ]
          ^                               ^
          arr[0]                         arr[4]

array[2] = 37 → acesso direto, O(1), instantâneo!
```

Busca Linear — O(n)

Percorre todos os elementos um a um até encontrar o que procura. Funciona em qualquer array, mesmo sem ordenação.

```
public class BuscaLinear {
    // Retorna o índice do elemento, ou -1 se não encontrar
    public static int buscar(int[] array, int alvo) {
        for (int i = 0; i < array.length; i++) {
            if (array[i] == alvo) {
                return i; // encontrado na posição i
            }
        }
        return -1; // não encontrado
    }

    public static void main(String[] args) {
        int[] numeros = {15, 42, 8, 97, 23, 56};

        int pos = buscar(numeros, 97);
        System.out.println("97 encontrado na posição: " + pos); // 3

        pos = buscar(numeros, 100);
        System.out.println("100 encontrado na posição: " + pos); // -1
    }
}
```

Quando usar Busca Linear?

- Array pequeno (menos de ~100 elementos)
- Array não está ordenado
- Você precisa fazer apenas uma busca

Para arrays grandes e ordenados, sempre prefira a Busca Binária!

Busca Binária — O(log n)

Divide o array ao meio a cada passo. Funciona APENAS em arrays ordenados, mas é muito mais rápida: em 1 milhão de elementos, encontra qualquer valor em no máximo 20 comparações!

```
Array ordenado: [3][8][15][22][31][45][67][84][92][99]
Procurando: 45

Passo 1: meio = índice 4 → valor 31 → 45 > 31, vai para direita
          [3][8][15][22][31] | [45][67][84][92][99]

Passo 2: meio = índice 7 → valor 84 → 45 < 84, vai para esquerda
          [45][67][84]

Passo 3: meio = índice 5 → valor 45 → ENCONTRADO! 🎯
Total: 3 comparações para 10 elementos (vs 6 na busca linear)
```

```
public class BuscaBinaria {
    // REQUISITO: array deve estar ORDENADO!
    public static int buscar(int[] array, int alvo) {
        int inicio = 0;
        int fim = array.length - 1;

        while (inicio <= fim) {
            int meio = (inicio + fim) / 2;

            if (array[meio] == alvo) {
                return meio; // encontrado!
            } else if (array[meio] < alvo) {
                inicio = meio + 1; // vai para a metade direita
            } else {
                fim = meio - 1; // vai para a metade esquerda
            }
        }
        return -1; // não encontrado
    }

    public static void main(String[] args) {
        int[] ordenado = {3, 8, 15, 22, 31, 45, 67, 84, 92, 99};

        System.out.println(buscar(ordenado, 45)); // 5
        System.out.println(buscar(ordenado, 50)); // -1
    }
}
```

Operação	Melhor	Médio	Pior
Busca Linear	$O(1)$	$O(n)$	$O(n)$
Busca Binária	$O(1)$	$O(\log n)$	$O(\log n)$
Acesso por índice	$O(1)$	$O(1)$	$O(1)$
Inserção no fim	$O(1)$	$O(1)$	$O(1)$
Inserção no meio	$O(n)$	$O(n)$	$O(n)$

7.2 Lista Ligada (Linked List)

Diferente do array, os elementos de uma lista ligada não ficam lado a lado na memória. Cada elemento (nó) guarda seu valor e um apontador para o próximo nó. Isso torna inserções e remoções muito mais eficientes.

Singly Linked List

Cada nó aponta apenas para o PRÓXIMO.

- Mais simples e usa menos memória
- Traversal só para frente
- Inserção/remoção no início: $O(1)$
- Remoção de nó específico: precisa do nó anterior

Doubly Linked List

Cada nó aponta para o PRÓXIMO e para o ANTERIOR.

- Traversal para frente e para trás
- Remoção de qualquer nó: $O(1)$ com referência
- Usa mais memória (ponteiro extra)
- Base para deque, LRU cache

SINGLY LINKED LIST:

```
[Head]
|
[10 | *] → [25 | *] → [37 | *] → [null]
nó 1      nó 2      nó 3      (fim)
```

DOUBLY LINKED LIST:

```
[Head]                                [Tail]
|                                    |
[null | 10 | *] ↔ [* | 25 | *] ↔ [* | 37 | null]
nó 1      nó 2      nó 3
```

Implementando uma Singly Linked List:

```
public class ListaLigada {

    // Classe interna que representa cada nó
    private static class No {
        int valor;
        No proximo;

        No(int valor) {
            this.valor = valor;
            this.proximo = null;
        }
    }

    private No cabeca; // primeiro nó da lista
    private int tamanho;

    // Inserir no início -  $O(1)$ 
    public void inserirInicio(int valor) {
        No novo = new No(valor);
        novo.proximo = cabeca;
        cabeca = novo;
        tamanho++;
    }

    // Inserir no fim -  $O(n)$ 
```

```

public void inserirFim(int valor) {
    No novo = new No(valor);
    if (cabeca == null) { cabeca = novo; tamanho++; return; }
    No atual = cabeca;
    while (atual.proximo != null) atual = atual.proximo;
    atual.proximo = novo;
    tamanho++;
}

// Remover do início - O(1)
public int removerInicio() {
    if (cabeca == null) throw new RuntimeException("Lista vazia!");
    int valor = cabeca.valor;
    cabeca = cabeca.proximo;
    tamanho--;
    return valor;
}

// Buscar um valor - O(n)
public boolean contem(int valor) {
    No atual = cabeca;
    while (atual != null) {
        if (atual.valor == valor) return true;
        atual = atual.proximo;
    }
    return false;
}

// Exibir todos os nós
public void exibir() {
    No atual = cabeca;
    while (atual != null) {
        System.out.print(atual.valor + " → ");
        atual = atual.proximo;
    }
    System.out.println("null");
}

public static void main(String[] args) {
    ListaLigada lista = new ListaLigada();
    lista.inserirFim(10);
    lista.inserirFim(25);
    lista.inserirFim(37);
    lista.inserirInicio(5);
    lista.exibir(); // 5 → 10 → 25 → 37 → null
    System.out.println("Contém 25? " + lista.contem(25)); // true
}

```

Operação	Melhor	Médio	Pior
Acesso por índice	$O(n)$	$O(n)$	$O(n)$
Busca por valor	$O(1)$	$O(n)$	$O(n)$
Inserção no início	$O(1)$	$O(1)$	$O(1)$
Inserção no fim	$O(1) *$	$O(n)$	$O(n)$
Remoção no início	$O(1)$	$O(1)$	$O(1)$

* $O(1)$ no fim se você mantiver um ponteiro para o último nó (tail pointer).

7.3 Pilha (Stack) — LIFO

A pilha segue o princípio LIFO: Last In, First Out (último a entrar, primeiro a sair). Pense em uma pilha de pratos: você só coloca e retira pelo topo.

Aplicações Reais

- Histórico do navegador (botão Voltar)
- Ctrl+Z (desfazer operações)
- Chamadas de funções (call stack)
- Verificar parênteses balanceados
- Avaliação de expressões matemáticas
- Algoritmos de backtracking

Operações Principais

- `push(x)` — empilha elemento no topo
- `pop()` — desempilha e retorna o topo
- `peek()` — espia o topo sem remover
- `isEmpty()` — verifica se está vazia
- `size()` — retorna o tamanho

Todas as operações são $O(1)$!

PILHA — opera sempre pelo TOPO

<code>push(10)</code>	<code>push(25)</code>	<code>push(37)</code>	<code>pop()</code>	<code>peek()</code>
		<code>[37] ← topo</code>		
	<code>[25]</code>	<code>[25]</code>	<code>[25] ← topo</code>	
<code>[10]</code>	<code>[10]</code>	<code>[10]</code>	<code>[10]</code>	<code>[10]</code>
base	base	base	base	base
			retorna 37	retorna 25

Usando Stack em Java:

```
import java.util.Stack;
import java.util.Deque;
import java.util.ArrayDeque;

public class ExemploStack {
    public static void main(String[] args) {
        // Recomendado: use Deque em vez de Stack (mais moderno)
        Deque<Integer> pilha = new ArrayDeque<>();

        // push — empilha
        pilha.push(10);
        pilha.push(25);
        pilha.push(37);
        System.out.println("Pilha: " + pilha); // [37, 25, 10]

        // peek — espia sem remover
        System.out.println("Topo: " + pilha.peek()); // 37

        // pop — desempilha
        System.out.println("Removido: " + pilha.pop()); // 37
        System.out.println("Pilha: " + pilha);          // [25, 10]

        System.out.println("Vazia? " + pilha.isEmpty()); // false
    }
}
```

```
}  
}
```

Aplicação prática: verificar parênteses balanceados

```
public static boolean balanceado(String expressao) {  
    Deque<Character> pilha = new ArrayDeque<>();  
  
    for (char c : expressao.toCharArray()) {  
        if (c == '(' || c == '[' || c == '{') {  
            pilha.push(c); // empilha os que abrem  
        } else if (c == ')' || c == ']' || c == '}') {  
            if (pilha.isEmpty()) return false;  
            char topo = pilha.pop();  
            if (c == ')' && topo != '(') return false;  
            if (c == ']' && topo != '[') return false;  
            if (c == '}' && topo != '{') return false;  
        }  
    }  
    return pilha.isEmpty(); // balanceado se pilha vazia  
}  
  
// balanceado("(a + [b * c])") → true  
// balanceado("{}{}") → false
```

7.4 Fila (Queue) — FIFO

A fila segue o princípio FIFO: First In, First Out (primeiro a entrar, primeiro a sair). Exatamente como uma fila de banco: quem chega primeiro é atendido primeiro.

Aplicações Reais

- Fila de atendimento ao cliente
- Fila de impressão
- Processos do sistema operacional
- Streaming de dados (buffer)
- Algoritmo BFS em grafos
- Sistema de mensagens (message queue)

Operações Principais

- offer(x) — enfileira no final
- poll() — desenfileira do início
- peek() — espia o primeiro sem remover
- isEmpty() — verifica se está vazia
- size() — retorna o tamanho

Todas as operações são O(1)!

FILA — entra no FIM, sai do INÍCIO

```
offer(10) offer(25) offer(37) poll() peek()  
INÍCIO FIM INÍCIO FIM INÍCIO FIM  
[10] [10][25] [10][25][37]  
poll() → remove 10 (início)  
peek() → espia 25 (sem remover)
```

Usando Queue em Java:

```
import java.util.Queue;
```



```

import java.util.LinkedList;

public class ExemploQueue {
    public static void main(String[] args) {
        Queue<String> fila = new LinkedList<>();

        // offer - enfileira
        fila.offer("Cliente A");
        fila.offer("Cliente B");
        fila.offer("Cliente C");
        System.out.println("Fila: " + fila);

        // peek - primeiro sem remover
        System.out.println("Próximo: " + fila.peek()); // Cliente A

        // poll - desenfileira
        System.out.println("Atendendo: " + fila.poll()); // Cliente A
        System.out.println("Atendendo: " + fila.poll()); // Cliente B
        System.out.println("Fila: " + fila);           // [Cliente C]
    }
}

```

Aplicação: simulação de atendimento

```

import java.util.*;

public class SistemaAtendimento {
    public static void main(String[] args) {
        Queue<String> fila = new LinkedList<>();
        Scanner sc = new Scanner(System.in);
        int opcao;

        do {
            System.out.println("\n1-Entrar na fila  2-Atender  3-Ver fila  4-Sair");
            opcao = sc.nextInt(); sc.nextLine();

            switch (opcao) {
                case 1:
                    System.out.print("Nome do cliente: ");
                    fila.offer(sc.nextLine());
                    System.out.println("Adicionado! Posição: " + fila.size());
                    break;
                case 2:
                    if (!fila.isEmpty())
                        System.out.println("Atendendo: " + fila.poll());
                    else System.out.println("Fila vazia!");
                    break;
                case 3:
                    System.out.println("Fila atual: " + fila);
                    break;
            }
        } while (opcao != 4);
    }
}

```

Estruturas Não Lineares

O que são estruturas não lineares?

Nas estruturas lineares (array, lista, pilha, fila), os elementos formam uma sequência. Nas não lineares, cada elemento pode se conectar a múltiplos outros — criando hierarquias, redes e mapas de dados.

8.1 Hash Table — Busca em $O(1)$

Uma Hash Table converte uma chave (como uma string) em um índice numérico usando uma função de hash. Isso permite busca, inserção e remoção em tempo $O(1)$ no caso médio — praticamente instantâneo.

Como funciona a Hash Table:

```
chave "Ana" → hash("Ana") = 42 → tabela[42] = "Ana: 99998888"
chave "Bruno" → hash("Bruno") = 7 → tabela[7] = "Bruno: 98887777"
```

```
Busca: get("Ana") → hash("Ana") = 42 → tabela[42] →  $O(1)$  !
```

COLISÃO: duas chaves geram o mesmo índice

```
hash("Carlos") = 42 → mesma posição de "Ana"!
```

Solução: encadeamento (lista ligada na mesma posição)

```
tabela[42] → [Ana:99998888] → [Carlos:97776666] → null
```

HashMap em Java — já é uma Hash Table:

```
import java.util.HashMap;
import java.util.Map;

public class HashTableDemo {
    public static void main(String[] args) {
        // HashMap é a implementação de Hash Table do Java
        Map<String, String> agenda = new HashMap<>();

        // put —  $O(1)$  amortizado
        agenda.put("Ana", "(79) 99999-1111");
        agenda.put("Bruno", "(79) 98888-2222");
        agenda.put("Carlos", "(11) 97777-3333");

        // get —  $O(1)$  amortizado
        String tel = agenda.get("Bruno");
        System.out.println("Bruno: " + tel);

        // containsKey —  $O(1)$ 
        System.out.println(agenda.containsKey("Ana")); // true

        // Frequência de caracteres — uso clássico de HashMap
        String texto = "estrutura de dados";
        Map<Character, Integer> freq = new HashMap<>();
        for (char c : texto.toCharArray()) {
            freq.put(c, freq.getOrDefault(c, 0) + 1);
        }
        System.out.println(freq); // {e=2, s=2, t=2, ...}
    }
}
```

Operação	Melhor	Médio	Pior
Inserção (put)	$O(1)$	$O(1)$	$O(n)^*$
Busca (get)	$O(1)$	$O(1)$	$O(n)^*$
Remoção (remove)	$O(1)$	$O(1)$	$O(n)^*$
Iteração	$O(n)$	$O(n)$	$O(n)$

* $O(n)$ no pior caso ocorre quando há muitas colisões. Com uma boa função de hash, isso é raro.

8.2 Árvores (Trees)

Uma árvore é uma estrutura hierárquica onde cada elemento (nó) pode ter filhos. A raiz fica no topo, as folhas no final das ramificações.

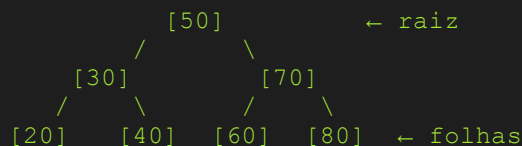
Terminologia essencial:

- Raiz (root) — nó principal, sem pai
- Filho (child) — nó conectado abaixo de outro
- Pai (parent) — nó que aponta para o filho
- Folha (leaf) — nó sem filhos
- Altura — distância da raiz até a folha mais profunda
- Nível — distância de um nó até a raiz

Binary Search Tree (BST) — Árvore Binária de Busca

Na BST, cada nó tem no máximo 2 filhos. A regra fundamental: valores menores vão para a ESQUERDA, valores maiores vão para a DIREITA. Isso permite busca em $O(\log n)$ numa árvore balanceada.

Inserindo: 50, 30, 70, 20, 40, 60, 80



Busca por 40:

50 → 40 < 50, vai esquerda

30 → 40 > 30, vai direita

40 → ENCONTRADO! (3 comparações para 7 elementos)

```

public class BST {
    private static class No {
        int valor;
        No esquerda, direita;
        No(int v) { valor = v; }
    }

    private No raiz;

    // INSERÇÃO - O(log n) em árvore balanceada
    public void inserir(int valor) {
        raiz = inserirRec(raiz, valor);
    }

    private No inserirRec(No no, int valor) {
        if (no == null) return new No(valor); // posição encontrada
        if (valor < no.valor) no.esquerda = inserirRec(no.esquerda, valor);
        else if (valor > no.valor) no.direita = inserirRec(no.direita, valor);
        return no;
    }

    // BUSCA - O(log n) em árvore balanceada
    public boolean buscar(int valor) {
        return buscarRec(raiz, valor);
    }

    private boolean buscarRec(No no, int valor) {
        if (no == null) return false;
        if (valor == no.valor) return true;
        if (valor < no.valor) return buscarRec(no.esquerda, valor);
        return buscarRec(no.direita, valor);
    }

    // PERCURSO In-Ordem (esq → raiz → dir) = imprime ordenado!
    public void inOrdem() {
        inOrdemRec(raiz);
        System.out.println();
    }

    private void inOrdemRec(No no) {
        if (no == null) return;
        inOrdemRec(no.esquerda);
        System.out.print(no.valor + " ");
        inOrdemRec(no.direita);
    }

    public static void main(String[] args) {
        BST arvore = new BST();
        for (int v : new int[]{50, 30, 70, 20, 40, 60, 80}) arvore.inserir(v);

        arvore.inOrdem(); // 20 30 40 50 60 70 80 (ordenado!)
        System.out.println(arvore.buscar(40)); // true
        System.out.println(arvore.buscar(99)); // false
    }
}

```

Os 3 percursos de uma árvore binária:

Percurso	Ordem de visita	Resultado no exemplo	Uso típico
----------	-----------------	----------------------	------------

In-Ordem	Esq → Raiz → Dir	20 30 40 50 60 70 80	Imprimir ordenado
Pré-Ordem	Raiz → Esq → Dir	50 30 20 40 70 60 80	Copiar/serializar árvore
Pós-Ordem	Esq → Dir → Raiz	20 40 30 60 80 70 50	Deletar árvore, expressões

Operação	Melhor	Médio	Pior
Busca	$O(\log n)$	$O(\log n)$	$O(n)^*$
Inserção	$O(\log n)$	$O(\log n)$	$O(n)^*$
Remoção	$O(\log n)$	$O(\log n)$	$O(n)^*$
Percurso	$O(n)$	$O(n)$	$O(n)$

* $O(n)$ no pior caso quando a árvore fica degenerada (inserção sempre ordenada). Árvores AVL e Red-Black resolvem isso automaticamente.

8.3 Heap — Prioridade Sempre no Topo

Um Heap é uma árvore binária com uma propriedade especial: o elemento de maior (Max Heap) ou menor (Min Heap) prioridade sempre fica na raiz. É a base de filas de prioridade e algoritmos de ordenação como o Heap Sort.

Max Heap O MAIOR valor sempre está na raiz. Cada pai é maior ou igual a seus filhos. • Extrair o máximo: $O(\log n)$ • Inserir novo elemento: $O(\log n)$ • Ver o máximo: $O(1)$ Usado em: agendadores de tarefas, algoritmo de Dijkstra invertido.	Min Heap O MENOR valor sempre está na raiz. Cada pai é menor ou igual a seus filhos. • Extrair o mínimo: $O(\log n)$ • Inserir novo elemento: $O(\log n)$ • Ver o mínimo: $O(1)$ Usado em: Dijkstra, Prim, fila de prioridade padrão do Java.
--	--

MAX HEAP: <pre> [90] / \ [70] [80] / \ / \ [40][60][50][30] </pre> Propriedade: pai >= filhos Raiz = sempre o maior	MIN HEAP: <pre> [5] / \ [12] [10] / \ / \ [20][15][25][18] </pre> Propriedade: pai <= filhos Raiz = sempre o menor
---	--

PriorityQueue — Heap no Java:

```
import java.util.PriorityQueue;
import java.util.Collections;

public class ExemploHeap {
    public static void main(String[] args) {

        // MIN HEAP (padrão no Java — menor elemento sempre no topo)
        PriorityQueue<Integer> minHeap = new PriorityQueue<>();
        minHeap.offer(30);
        minHeap.offer(10);
        minHeap.offer(50);
        minHeap.offer(5);
        minHeap.offer(20);

        System.out.print("Min Heap (extraíndo em ordem): ");
        while (!minHeap.isEmpty()) {
            System.out.print(minHeap.poll() + " ");
        } // 5 10 20 30 50 — sempre extrai o menor!

        // MAX HEAP — usar Collections.reverseOrder()
        PriorityQueue<Integer> maxHeap = new
PriorityQueue<>(Collections.reverseOrder());
        maxHeap.offer(30);
        maxHeap.offer(10);
        maxHeap.offer(50);
        maxHeap.offer(5);
        maxHeap.offer(20);

        System.out.print("Max Heap (extraíndo em ordem): ");
        while (!maxHeap.isEmpty()) {
            System.out.print(maxHeap.poll() + " ");
        } // 50 30 20 10 5 — sempre extrai o maior!

        // Fila de prioridade de tarefas
        PriorityQueue<String> tarefas = new PriorityQueue<>();
        tarefas.offer("C - Reunião de equipe");
        tarefas.offer("A - Bug crítico em produção");
        tarefas.offer("B - Code review pendente");

        System.out.println("\nPróxima tarefa: " + tarefas.peek());
        // A - Bug crítico (ordem alfabética = menor primeiro)
    }
}
```

8.4 Grafos (Graphs)

Grafos são a estrutura mais geral e poderosa. Diferente de árvores, não têm raiz nem hierarquia obrigatória — são redes de nós (vértices) conectados por arestas. Qualquer rede do mundo real pode ser modelada como um grafo.

Exemplos de grafos no mundo real:

- Redes sociais (usuários = vértices, amizades = arestas)
- Google Maps (cidades = vértices, estradas = arestas com peso = distância)

- Internet (servidores = vértices, conexões = arestas)
- LinkedIn (profissionais = vértices, conexões = arestas)
- Dependências de projetos (tarefas = vértices, dependências = arestas)

Terminologia dos Grafos:

Conceito	Definição	Exemplo
Vértice (Vertex)	Nó do grafo	Cidade, usuário, servidor
Aresta (Edge)	Conexão entre vértices	Estrada, amizade, link
Grafo Direcionado	Arestas têm sentido ($\rightarrow$)	Twitter: você segue alguém
Grafo Não Direcionado	Arestas são bidirecionais	Facebook: amizade mútua
Grafo Ponderado	Arestas têm peso/custo	Distância entre cidades
Grau (Degree)	Número de arestas de um vértice	Quantos amigos você tem
Ciclo	Caminho que volta ao início	$A \rightarrow B \rightarrow C \rightarrow A$
Caminho	Sequência de vértices conectados	$A \rightarrow B \rightarrow C \rightarrow D$

GRAFO NÃO DIRECIONADO:



GRAFO DIRECIONADO:



Representação por Lista de Adjacência:

```

A: [B, C]
B: [A, C, D]
C: [A, B, D]
D: [B, C]

```

Representação em Java:

```

import java.util.*;

public class Grafo {
    private Map<String, List<String>> adjacencia = new HashMap<>();

    // Adicionar vértice
    public void addVertice(String v) {
        adjacencia.putIfAbsent(v, new ArrayList<>());
    }

    // Adicionar aresta não direcionada
    public void addAresta(String v1, String v2) {
        adjacencia.get(v1).add(v2);
        adjacencia.get(v2).add(v1);
    }
}

```

```

// Exibir grafo
public void exibir() {
    adjacencia.forEach((v, vizinhos) ->
        System.out.println(v + " → " + vizinhos)
    );
}

public static void main(String[] args) {
    Grafo g = new Grafo();
    for (String v : new String[]{"A","B","C","D"}) g.addVertice(v);
    g.addAresta("A", "B");
    g.addAresta("A", "C");
    g.addAresta("B", "D");
    g.addAresta("C", "D");
    g.exibir();
    // A → [B, C]
    // B → [A, D]
    // C → [A, D]
    // D → [B, C]
}
}

```

DFS — Busca em Profundidade (Depth-First Search)

A DFS mergulha fundo em um caminho antes de voltar e explorar outros. Usa uma pilha (ou recursão). Boa para detectar ciclos, encontrar caminhos, e resolver labirintos.

DFS a partir de A:

Grafo: A—B—D
 | |
 C—E

Passo 1: visita A → pilha=[A], visitados={A}
 Passo 2: vizinho B → pilha=[A,B], visitados={A,B}
 Passo 3: vizinho D → pilha=[A,B,D], visitados={A,B,D}
 Passo 4: volta → vizinho E → pilha=[A,B,E], visitados={A,B,D,E}
 Passo 5: volta → vizinho C → visitados={A,B,D,E,C}
 Ordem de visita: A → B → D → E → C

```

// DFS com recursão
public void dfs(String inicio) {
    Set<String> visitados = new HashSet<>();
    dfsRec(inicio, visitados);
    System.out.println();
}

private void dfsRec(String vertice, Set<String> visitados) {
    visitados.add(vertice);
    System.out.print(vertice + " ");

    for (String vizinho : adjacencia.get(vertice)) {
        if (!visitados.contains(vizinho)) {
            dfsRec(vizinho, visitados);
        }
    }
}
}

```


BFS — Busca em Largura (Breadth-First Search)

A BFS visita todos os vizinhos diretos antes de ir mais fundo. Usa uma fila. Garante encontrar o caminho mais curto em grafos sem peso.

BFS a partir de A:

Grafo: A—B—D
| |
C—E

Nível 0: A

Nível 1: B, C (vizinhos diretos de A)

Nível 2: D, E (vizinhos de B e C ainda não visitados)

Ordem de visita: A → B → C → D → E

Distâncias a partir de A: B=1, C=1, D=2, E=2

```
// BFS com fila
public void bfs(String inicio) {
    Set<String> visitados = new HashSet<>();
    Queue<String> fila = new LinkedList<>();

    visitados.add(inicio);
    fila.offer(inicio);

    while (!fila.isEmpty()) {
        String atual = fila.poll();
        System.out.print(atual + " ");

        for (String vizinho : adjacencia.get(atual)) {
            if (!visitados.contains(vizinho)) {
                visitados.add(vizinho);
                fila.offer(vizinho);
            }
        }
    }
    System.out.println();
}
```

Quando usar DFS?

- Detectar ciclos no grafo
- Resolver labirintos
- Topological Sort
- Componentes fortemente conectados
- Backtracking (sudoku, N-Rainhas)

Quando usar BFS?

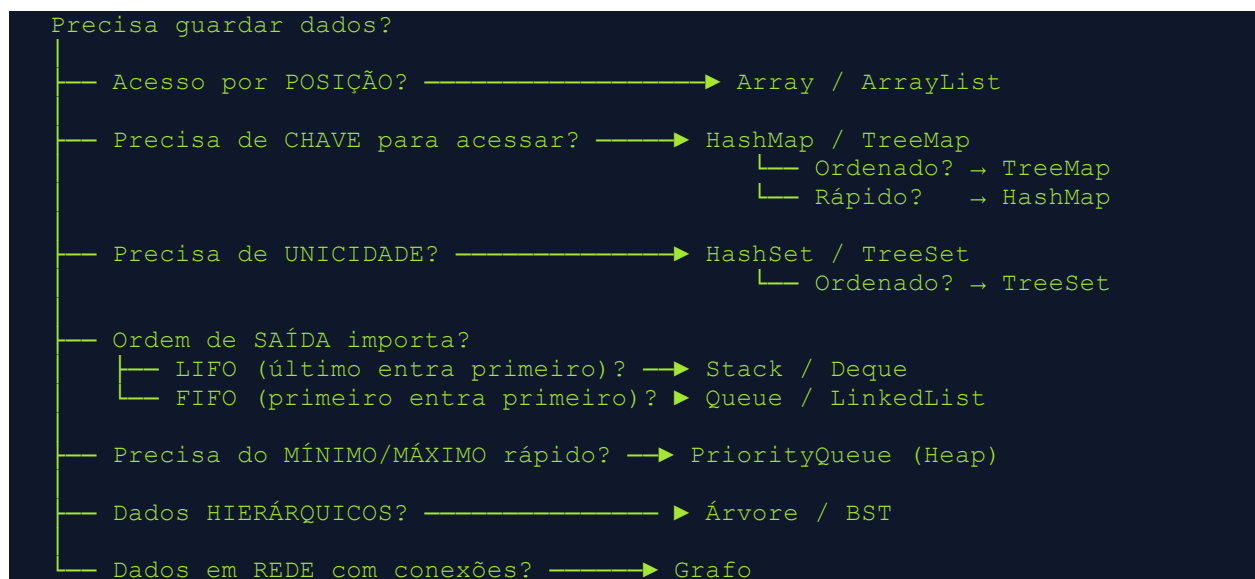
- Caminho mais curto (sem peso)
- Nível de separação entre nós
- Verificar se grafo é bipartido
- Web crawling por camadas
- Difusão em redes (contágio, rumores)

Qual Estrutura Usar?

Escolher a estrutura certa é a chave para um código eficiente. Use esta tabela como referência rápida:

Estrutura	Melhor para...	Cuidado com...	Complexidade chave
Array	Acesso por índice, dados fixos	Tamanho fixo, inserção no meio	Acesso $O(1)$
ArrayList	Lista dinâmica, acesso por índice	Inserção/remoção no meio $O(n)$	get $O(1)$, add $O(1)^*$
LinkedList	Inserções/remoções frequentes	Acesso por índice $O(n)$	insert $O(1)$, get $O(n)$
Stack	LIFO: undo, recursão, parsing	Não adequado para acesso aleatório	push/pop $O(1)$
Queue	FIFO: filas, BFS, streaming	Não adequado para acesso aleatório	offer/poll $O(1)$
HashMap	Busca por chave, contagem	Sem ordem, colisões no pior caso	get/put $O(1)$
TreeMap	Dados ordenados por chave	Mais lento que HashMap	get/put $O(\log n)$
HashSet	Elementos únicos, busca rápida	Sem ordem garantida	contains $O(1)$
BST	Dados ordenados + busca eficiente	Desbalanceamento $\rightarrow O(n)$	busca $O(\log n)^*$
Heap/PriorityQueue	Mínimo/máximo sempre disponível	Sem acesso aleatório	peek $O(1)$, poll $O(\log n)$
Grafo (adj. list)	Redes, caminhos, conexões	Memória proporcional a arestas	DFS/BFS $O(V+E)$

Árvore de Decisão Rápida



Lista de Exercícios

Consolide o aprendizado implementando e analisando:

Exercício 1: Verificador de Palíndromo com Stack

Use uma Stack para verificar se uma palavra é palíndromo (lê igual de trás para frente). Ex: "arara" → true, "casa" → false.

```
public static boolean ehPalindromo(String palavra) {
    Deque<Character> pilha = new ArrayDeque<>();
    for (char c : palavra.toCharArray()) pilha.push(c);

    for (char c : palavra.toCharArray()) {
        if (c != pilha.pop()) return false;
    }
    return true;
}
// ehPalindromo("arara") → true
// ehPalindromo("java") → false
```

Exercício 2: Fila de Impressão com Prioridade

Crie uma fila de impressão onde documentos urgentes (priority=1) são impressos antes dos normais (priority=2). Use PriorityQueue.

```
import java.util.*;

public class FilaImpressao {
    static class Documento implements Comparable<Documento> {
        String nome; int prioridade;
        Documento(String nome, int prior) { this.nome=nome; prioridade=prior; }
        public int compareTo(Documento d) { return this.prioridade - d.prioridade; }
        public String toString() { return "[P"+prioridade+"] "+nome; }
    }

    public static void main(String[] args) {
        PriorityQueue<Documento> fila = new PriorityQueue<>();
        fila.offer(new Documento("Relatorio.pdf", 2));
        fila.offer(new Documento("Contrato URGENTE.pdf", 1));
        fila.offer(new Documento("Apresentacao.pptx", 2));
        fila.offer(new Documento("NF Urgente.pdf", 1));
        while (!fila.isEmpty()) System.out.println("Imprimindo: " + fila.poll());
    }
}
```

Exercício 3: Busca em BST com Validação

Implemente uma BST que armazene nomes de alunos (String) e seus RAs (int) como chave. Insira 5 alunos, faça buscas e exiba em ordem alfabética usando percurso in-ordem.

```
// Adapte a classe BST para usar String como chave
```

```
// e armazenar pares (nome, RA)
public class BSTAlunos {
    private static class No {
        String nome; int ra;
        No esq, dir;
        No(String n, int r) { nome=n; ra=r; }
    }
    private No raiz;

    public void inserir(String nome, int ra) {
        raiz = inserirRec(raiz, nome, ra);
    }
    private No inserirRec(No n, String nome, int ra) {
        if (n == null) return new No(nome, ra);
        int cmp = nome.compareTo(n.nome);
        if (cmp < 0) n.esq = inserirRec(n.esq, nome, ra);
        else if (cmp > 0) n.dir = inserirRec(n.dir, nome, ra);
        return n;
    }
    public void inOrdem(No n) {
        if (n==null) return;
        inOrdem(n.esq);
        System.out.println(n.ra + " - " + n.nome);
        inOrdem(n.dir);
    }
}
}
```

Exercício 4: Grafos — Rede de Amizades

Modele uma rede social com 5 pessoas. Adicione amizades e implemente: listar amigos de uma pessoa, verificar se duas pessoas são amigas, e encontrar todos os alcançáveis a partir de uma pessoa (use BFS).

```
// Monte o grafo e complete os métodos
Grafo rede = new Grafo();
for (String p : new String[]{"Ana", "Bruno", "Carlos", "Diana", "Elisa"})
    rede.addVertice(p);
rede.addAresta("Ana", "Bruno");
rede.addAresta("Ana", "Carlos");
rede.addAresta("Bruno", "Diana");
rede.addAresta("Carlos", "Elisa");

// Implementar:
// 1. listarAmigos("Ana") → [Bruno, Carlos]
// 2. saoAmigos("Ana", "Diana") → false (amigos de 2º grau)
// 3. alcancaveis("Ana") → BFS: Ana Bruno Carlos Diana Elisa
```

Você dominou Arrays, listas, pilhas, filas, árvores, heaps e grafos. Continue praticando!